4. Architektura gry i sztucznej inteligencji

Jednym z głównych celów pracy jest stworzenie uproszczonej wersji gry *Heroes of Might and Magic 3*, oraz sztucznej inteligencji, która byłaby zdolna do sterowania agentem komputerowym w tej grze. W ramach tej pracy, stworzono dwa programy: jeden będący przystosowaną dla użytkownika grą komputerową, a drugi będący programem do trenowania sztucznej inteligencji wykorzystywanej w środowisku gry.

4.1. Założenia projektowe

4.1.1. Założenia projektowe gry

W ramach tworzenia uproszczonej wersji gry *HOMM3*, zdecydowano się na przyjęcie poniższych założeń projektowych:

- Stworzona gra powinna jak najwierniej odwzorowywać oryginalną grę *HOMM3*.
- System zarządzania i sterowania zasobami gracza powinien być stworzony w taki sposób, aby jednocześnie mogli z niego korzystać zarówno gracz komputerowy, jak i użytkownik gry.
- Środowisko powinno aktualizować stan gry za każdym razem, gdy gracz dowolnego typu wykona akcję tego wymagającą.
- Gra powinna być stworzona w taki sposób, aby każdy rodzaj gracza miał do dyspozycji informację o stanie gry, na jak najbardziej podobnych zasadach, które są w oryginalnej grze. W przypadku użytkownika gry, taka informacja powinna być przekazywana za pomocą graficznego interfejsu użytkownika.
- Silnik gry powinien być stworzony w taki sposób, aby, na potrzeby trenowania sztucznej inteligencji, możliwe było uruchomienie środowiska gry w sposób niewymagający tworzenia graficznego interfejsu użytkownika.

4.1.2. Założenia projektowe sztucznej inteligencji i programu trenującego

W ramach tworzenia sztucznej inteligencji do stworzonej gry, implementacji algorytmu NEAT, oraz programu trenującego komputerowych agentów, zdecydowano się na przyjęcie poniższych założeń projektowych:

- Implementacja algorytmu NEAT powinna być zgodna z oryginalną wersją algorytmu.
- Zarówno sieci neuronowe, jak i implementacja algorytmu NEAT, powinny być przystosowane do rozwiązywania dowolnego zadania.
- W celu podjęcia decyzji, komputerowy gracz może korzystać tylko z tych samych informacji o stanie gry, które posiada użytkownik.
- Sieci neuronowe powinny być w stanie poprawnie przetworzyć dane zawarte w stanie gry.
- W programie powinien istnieć interfejs, który umożliwia przetworzenie wyjścia sieci neuronowej na akcję w środowisku gry.

- Program trenujący sztuczną inteligencję musi wyznaczać wartości funkcji przystosowania trenowanych sieci neuronowych. W tym celu, musi on także pozyskiwać dane potrzebne do wyznaczania tych wartości.
- Program trenujący sztuczną inteligencję do gry, w celu realizacji swoich zadań (m.in. oceny graczy), może korzystać z zasobów silnika gry.

4.2. Wybór narzędzi

Jako główny język programowania wybrano język C++. Decyzja ta została podjęta ze względu na wcześniejszą znajomość i pracę z tym językiem. Język ten cechuje się także wysoką wydajnością, co jest istotne przy tworzeniu szybkiego programu do trenowania sztucznej inteligencji. W trakcie tworzenia pracy, w celu stworzenia testów jednostkowych, zastosowano bibliotekę do tworzenia testów — catch2[8].

Do tworzenia grafiki wykorzystywanej w grze, zastosowano bibliotekę Qt[9]. Qt jest zestawem narzędzi programistycznych, służących przede wszystkim do tworzenia graficznych interfejsów użytkownika. Decyzję tą podjęto na podstawie wcześniejszej znajomości tej biblioteki, dobrze opracowanej dokumentacji narzędzi, oraz prostej konstrukcji i obsłudze elementów graficznych zapewnianych przez tą bibliotekę.

4.3. Architektura gry

4.3.1. Elementy środowiska gry

Ze względu ma ograniczenia czasowe, gra zaprojektowana w ramach pracy posiada podzbiór najważniejszych elementów gry, które występują w oryginalnej wersji *HOMM3*. W stworzonej wersji gry, zostały zaimplementowane następujące elementy:

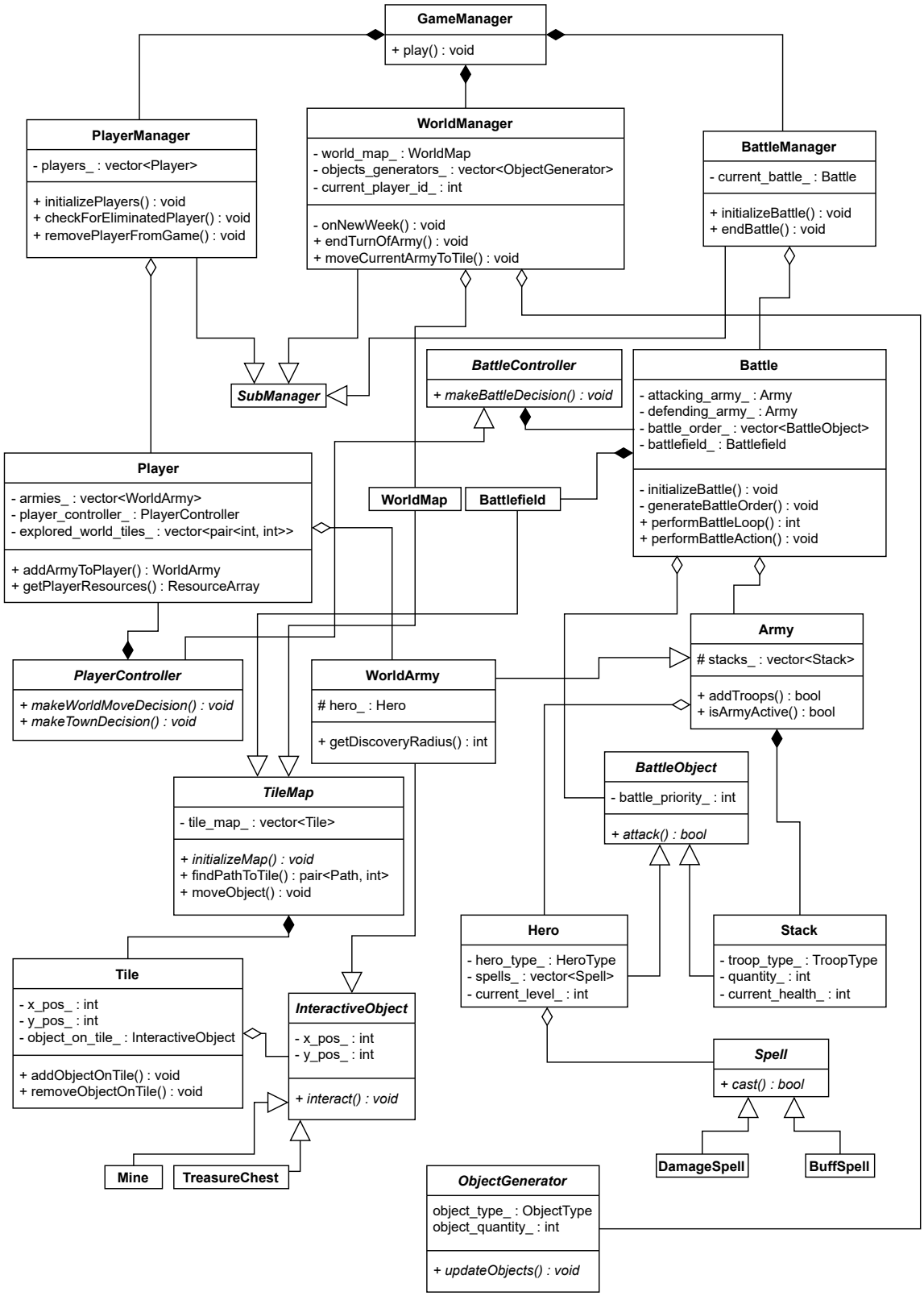
- Główna pętla gry, w której każdy z graczy, po kolei w swoich turach, eksploruje i odkrywa mapę świata za pomocą posiadanych armii.
- Możliwość wchodzenia w interakcję z obiektami na mapie świata, poprzez przesunięcie armii na pole sąsiadujące z obiektem; zaimplementowano obiekty opisane w rozdziale 2.6, a także stworzono system podejmowania decyzji związanych z tymi obiektami.
- Zasoby i poziomy bohatera, oraz systemy ich zdobywania.
- Wchodzenie w interakcję z wrogimi armiami, kończące się bitwą.
- Uproszczony, turowy system rozgrywania bitwy: kolejność wykonywania ruchów przez stosy zależy od ich szybkości, przy czym przy takiej samej wartości tego atrybutu, stosy armii atakującej wykonają swoje ruchy jako pierwsze. Bohaterowie także cechują się szybkością i podlegają turowemu systemowi bitwy (w oryginalnej wersji, gracz może podczas tury swojego stosu wykonać dodatkowo akcję bohaterem, przy czym gracz może wykonać jedną akcję powiązaną z bohaterem na rundę tj. jeden ruch wszystkimi stosami).
- Uproszczony system sterowania stosami:

  - W trakcie tury, gracz może przemieścić stos, zaatakować wroga, wykonać obydwie te akcje, albo pominąć turę. Stosy zadają punkty obrażeń wrogom zgodnie ze wzorem 1.
  - Pojedynczy stos posiada zbiorową pulę punktów zdrowia H, która na początku bitwy jest równa iloczynowi liczby jednostek w stosie i punktów zdrowia jednej jednostki. Wartość ta maleje za każdym razem, gdy stos otrzymuje obrażenia. Liczba jednostek w stosie jest zawsze równa $\left\lceil \frac{H}{t_h} \right\rceil$, gdzie t_h oznacza liczbę punktów zdrowia pojedynczej jednostki.
  - Gracze mogą sterować stosami w taki sposób, aby zablokować atak wrogiego stosu atakującego z dystansu. Aby stos mógł zaatakować wroga, między nim a celem nie może znajdować się nieprzekraczalne pole, ani jakakolwiek inny stos.
  - Stosy posiadają atrybuty opisane w rozdziale 2.3.2.
- Uproszczony system rzucania zaklęć przez bohaterów: gracz, w trakcie tury bohatera, wybiera jedno z zaklęć i wybiera stos lub pole, które będzie celem tego zaklęcia (alternatywnie, gracz może podjąć decyzję o pominięciu tury bohatera). Do uproszonej wersji gry dodano zaklęcia wzmacniające przyjazne jednostki poprzez zwiększenie ich statystyk i zaklęcia zadające obrażenia wrogom. Inny podział zaimplementowanych zaklęć to zaklęcia mające konkretny cel i zaklęcia działające obszarowo. We stworzonej wersji gry, bohater posiada także atrybuty (opisane w rozdziale 2.4), które modyfikują działanie zaklęć. Bohater dysponuje punktami many, które zmniejszają się po każdym rzuconym zaklęciu; brak wystarczającej liczby punktów many uniemożliwia dalszą możliwość rzucania zaklęć. Punkty many resetują się do początkowej wartości po każdej bitwie.
- Wchodzenie w interakcję z miastami: stworzono system odwiedzania przyjaznych miast i system zdobywania wrogich miast.
- Kupowanie w mieście zasobów dla swoich armii (jednostek, bohaterów i zaklęć).
- Możliwość rozbudowy swoich miast, poprzez ulepszanie budynków; zaimplementowano budynki opisane w rozdziale 2.7.
- System eliminacji gracza z rozgrywki z powodu braku kontroli nad przynajmniej jednym miastem lub jedną armią.

4.3.2. Główna pętla gry i system zarządzania graczami

Główną klasą gry jest klasa GameManager. Klasa ta jest odpowiedzialna za inicjację wszystkich elementów gry, uruchomienie pętli gry za pomocą metody *play*, oraz jest odpowiedzialna za zakończenie rozgrywki. W implementacji gry stworzono zestaw sześciu klas — menadżerów, których zadaniem jest obsługa i zarządzanie poszczególnymi elementami i systemami gry. Stworzono także szereg klas pomocniczych. Zależności pomiędzy nimi przedstawiono na rysunkach 4.1 i 4.4.

Klasa PlayerManager (rysunek 4.1) jest odpowiedzialna za zarządzanie graczami, two-



Rysunek 4.1. Diagram UML środowiska gry — klasy PlayerManager, WorldManager, BattleManager i klasy z nimi powiązane.

rzenie graczy podczas inicjacji, przydzielanie im zasobów podczas gry, oraz usuwanie graczy i ich zasobów po spełnieniu warunku eliminacji. Każdy gracz reprezentowany jest przez obiekt klasy Player. Obiekty tej klasy posiadają informację o stanie gracza i o posiadanych przez niego armiach, odkrytym przez jego armie terenie na mapie świata, a także o posiadanych przez niego zasobach (złoto, drewno, itp.). Maksymalnie w rozgrywce może uczestniczyć 4 graczy.

Armie posiadane przez gracza są obiektami klasy WorldArmy, które, oprócz możliwości podejmowania walki, mogą poruszać się po mapie świata. Każdy gracz może posiadać co najwyżej 4 armie.

Każdy z graczy posiada swój kontroler — obiekt klasy PlayerController. Klasa ta służy jako interfejs pomiędzy graczem, a elementami gry. Za każdym razem, kiedy gracz musi podjąć jakąś decyzję (np. przemieścić armię na mapie świata, albo kupić zasoby w mieście), system odpowiedzialny za dany element gry wywołuje odpowiednią metodę w klasie PlayerController, która pyta gracza jaką decyzję chce podjąć. PlayerController zwraca później systemowi podjętą przez gracza decyzję. Sposób odpytywania gracza zależy od jego typu. Dla użytkownika, klasa ta oczekuje na wejście gracza, które jest podawane za pomocą graficznego interfejsu użytkownika. Dla gracza komputerowego, uruchamiany jest zaimplementowany algorytm wyboru akcji.

4.3.3. System eksploracji świata

Klasa WorldManager (rysunek 4.1) jest odpowiedzialna za zarządzanie mapą świata. Klasa ta zarządza pętlą gry, polegającą na poruszaniu się armiami po mapie świata.

Mapa świata jest reprezentowana przez obiekt klasy WorldMap, która jest implementacją klasy TileMap. Obiekty klasy TileMap reprezentują pewną mapę, na której toczy się rozgrywka, a sama klasa udostępnia zestaw metod do pracy nad jej elementami — m.in. udostępnia metodę *findPathToTile*, która wyszukuje ścieżkę do danego pola za pomocą stworzonej implementacji algorytmu A*[10]. Klasa ta jest kontenerem dla klasy Tile, która reprezentuje pojedyncze pole na mapie. Obiekty klasy Tile posiadają informację o swoich koordynatach na mapie, oraz o obiekcie, który znajduje się na danym polu. Każdy obiekt, który może znajdować się na mapie świata i który może wchodzić w interakcję z graczami i ich armiami, dziedziczy po klasie InteractiveObject i implementuje metodę *interact*.

WorldManager tworzy obiekty klasy ObjectGenerator. Każdy z tych obiektów zarządza generacją i aktualizowaniem obiektów jednego typu znajdujących się na mapie świata (skrzyń ze skarbami, kopalń, itp.). Wszystkie obiekty, oprócz armii i miast, są zarządzane przez obiekty klasy ObjectGenerator.

4.3.4. System przeprowadzania bitew

Do zarządzania bitwami służy klasa BattleManager (rysunek 4.1). Bitwa może zostać zainicjowana, kiedy armia gracza wchodzi w interakcję z wrogą armią, lub z wrogim miastem. BattleManager tworzy wówczas obiekt klasy Battle. Klasa ta pobiera dane o dwóch walczą-



Rysunek 4.2. Zrzut ekranu przedstawiający fazę eksploracji świata.

cych armiach, inicjuje bitwę, oraz zarządza pętlą bitwy, (metoda *performBattleLoop*). Po bitwie, klasa Battle zwraca jej wynik, a klasa BattleManager informuje pozostałe systemy gry o wyniku tej bitwy.

W bitwach biorą udział armie reprezentowane przez obiekty klasy Army. Obiekty tej klasy są kontenerami dla stosów armii (obiektów klasy Stack), oraz opcjonalnie bohatera (obiektu klasy Hero). Instancja klasy Stack posiada informację o jednym stosie. Instancje klasy Hero posiadają informację o statystykach bohatera, oraz o posiadanych przez niego zaklęciach (obiektach klasy Spell). Wymienione wyżej klasy dziedziczą po klasie BattleObject, która umożliwia im przeprowadzenie akcji podczas bitwy za pomocą metody *attack*. Jedna armia może posiadać maksymalnie 8 różnych rodzajów stosów, a jej bohater może posiadać maksymalnie 4 zaklęcia.

Z każdą armią powiązany jest obiekt klasy BattleController. Klasa ta, podobnie jak PlayerController (PlayerController dziedziczy po BattleController), służy jako interfejs pomiędzy podjętymi decyzjami gracza, a przeprowadzaną bitwą.

4.3.5. System odwiedzania i ulepszania miast

TownManager (rysunek 4.4) zarządza miastami na mapie świata. Każde miasto jest reprezentowane przez obiekt klasy Town. Obiekty tej klasy posiadają informację o zasobach, jakie gracz kontrolujący miasto może kupić w mieście. Klasa Town jest także kontenerem



Rysunek 4.3. Zrzut ekranu przedstawiający fazę przeprowadzania bitwy.

dla obiektów klasy Building, które reprezentują budynki będące w mieście. Ulepszanie budynków powoduje ulepszenie zasobów dostępnych w mieście.

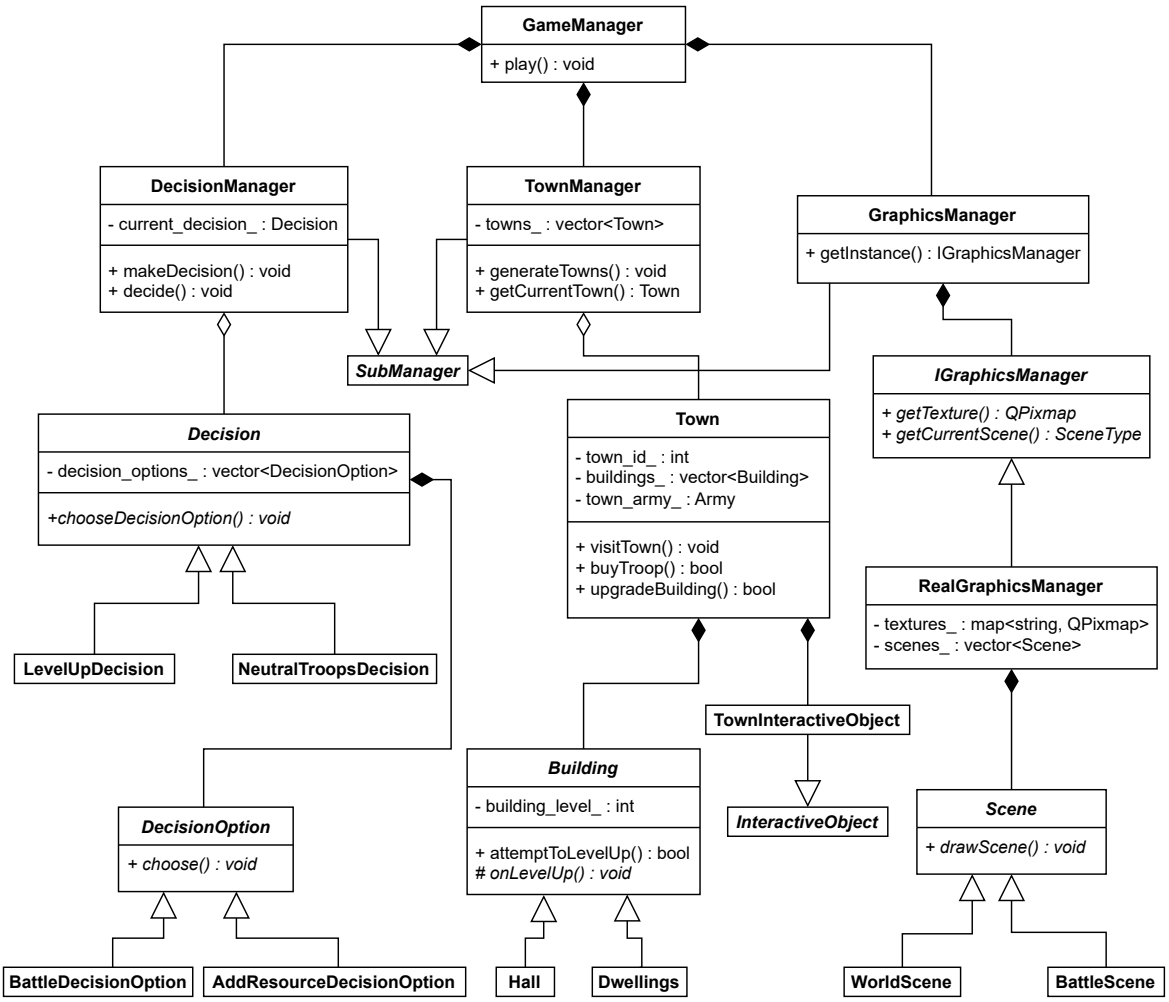
4.3.6. System podejmowania decyzji na mapie świata

DecisionManager (rysunek 4.4) zarządza decyzjami, jakie gracz może podjąć w takcie eksploracji świata. „Decyzja" w tym przypadku to sytuacja powstała w wyniku interakcji z mędrcem, skrzynią ze skarbami, lub neutralnymi jednostkami, albo w wyniku zwiększenia poziomu bohatera. Gracz wówczas musi wybrać jedną z dwóch lub trzech opcji podjęcia decyzji. Każda decyzja jest reprezentowana przez obiekt klasy Decision, a każda opcja możliwa do podjęcia — przez klasę DecisionOption. Wybór danego obiektu klasy DecisionOption powoduje określony skutek w grze.

4.3.7. Graficzny interfejs użytkownika

W środowisku gry, klasa GraphicsManager (rysunek 4.4) pełni rolę interfejsu pomiędzy graficznym interfejsem użytkownika, a pozostałymi elementami gry. Klasa ta zawiera jeden obiekt klasy IGraphicsManager. Istnieją dwa podejścia do tworzenia grafiki w programie, w zależności do wykorzystywanej implementacji tej klasy:

- Klasa RealGraphicsManager wykorzystuje bibliotekę graficzną Qt w celu tworzenia elementów graficznych wyświetlanych na ekranie. Każdej fazie gry przyporządkowany jest jeden obiekt klasy Scene. Do głównych scen gry należą: WorldScene



Rysunek 4.4. Diagram UML środowiska gry — klasy TownManager, DecisionManager, GraphicsManager i klasy z nimi powiązane.

(rysunek 4.2), BattleScene (rysunek 4.3) i TownScene. Klasa RealGraphicsManager wykorzystywana jest w aplikacji będącą grą.

- Klasa MockGraphicsManager nie wykorzystuje biblioteki graficznej, i nie implementuje logiki powiązanej z rysowaniem obiektów na ekranie. Klasa ta jest używana podczas trenowania sztucznej inteligencji, co umożliwia stworzenie środowiska gry bez tworzenia grafiki.

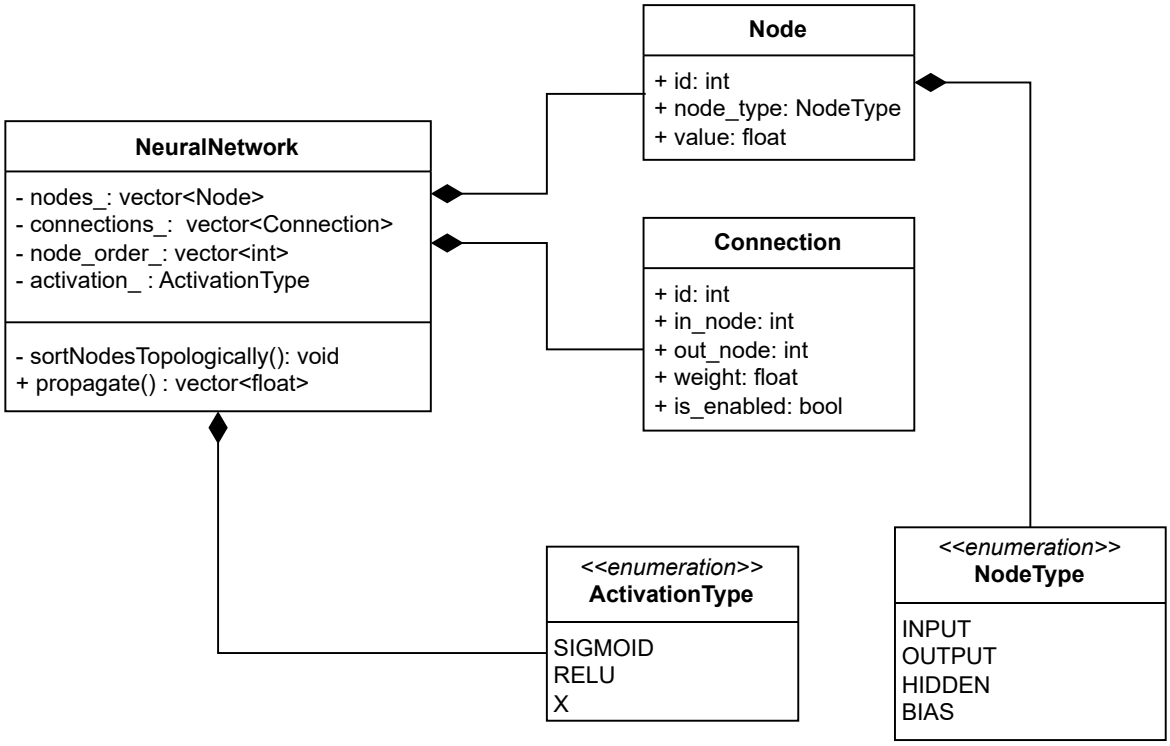
4.4. Architektura sztucznej inteligencji

4.4.1. Architektura sieci neuronowych

Sieci neuronowe w tej pracy reprezentowane są przez klasę NeuralNetwork. Obiekty tej klasy są wykorzystywane podczas tworzenia sztucznej inteligencji w implementacji algorytmu NEAT, jak i w środowisku gry do sterowania komputerowymi graczami.

Diagram klas i struktur powiązanych z sieciami neuronowymi znajduje się na rysunku 4.5. Obiekty klasy NeuralNetwork posiadają w sobie kontenery obiektów reprezentujące

połączenia (struktura Connection) i węzły (struktura Node) sieci. Sieć neuronowa reprezentowana jest przez graf skierowany. Klasa NeuralNetwork posiada wektor indeksów węzłów, oznaczający kolejność, w jakim węzły powinny być przetwarzane w trakcie propagacji w przód (listing 1). Za każdym razem, kiedy zmieniana jest struktura sieci, kolejność przetwarzania węzłów jest aktualizowana za pomocą algorytmu sortowania topologicznego Kahna. Implementacja algorytmu została oparta na otwartoźródłowym rozwiązaniu[11].



Rysunek 4.5. Diagram UML przedstawiający strukturę klas powiązanych ze sieciami neuronowymi.

Listing 1. Implementacja propagacji w przód w klasie NeuralNetwork.

```
1  vector<float> NeuralNetwork::propagate(vector<float>& input){
2  for (auto& node : nodes_) node.value = 0.0f;
3  for(int i=0; i<input.size(); ++i){
4      getNodeWithID(input_nodes_.at(i)).value = input.at(i);}
5  // Ostatnim węzłem wejściowym jest węzeł biasu
6  getNodeWithID(input_nodes_.back()).value = 1.0f;
7
8  for(const int& node_id: node_order_){
9      Node& node = getNodeWithID(node_id);
10     if(node.node_type == INPUT) continue;
11     float value = 0.0f;
12
13     for(const Connection& connection: connections_){
```

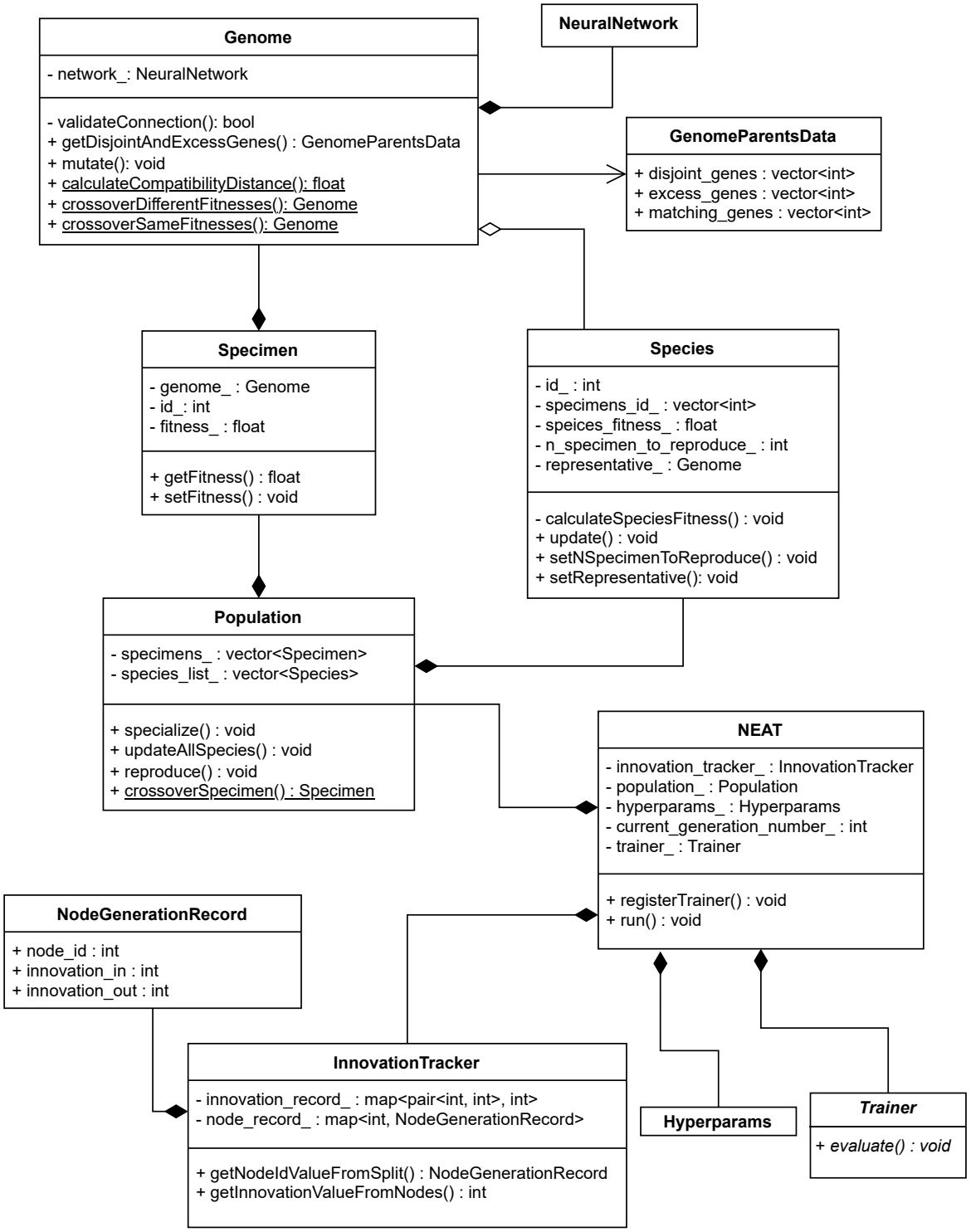
```
14            if(!connection.is_enabled) continue;
15            if(connection.out_node == node_id){
16                value += getNodeWithID(connection.in_node).value
17                    * connection.weight;}}
18        node.value = activation_(value);}
19
20  vector<float> output;
21  for(const int output_node: output_nodes_){
22        output.push_back(getNodeWithID(output_node).value);}
23  return output;
24  };
```

4.4.2. Implementacja algorytmu NEAT

Implementacja algorytmu NEAT w pracy składa się z następujących elementów:

- Obiekty klasy Genome zawierają obiekty klasy NeuralNetwork. Zadaniem tej klasy jest modyfikowanie sieci neuronowych, poprzez m.in. ich mutację, a także tworzenie potomstwa na bazie dwóch sieci.
- Obiekty klasy Specimen zawierają jeden obiekt klasy Genome. Obiekty te reprezentują osobników tworzonych podczas działania algorytmu, którzy podlegają ewaluacji.
- Gatunki są reprezentowane przez klasę Species. Obiekty tej klasy przechowują referencje do osobników należących do jednego gatunku. Zadaniem tej klasy jest aktualizowanie danych osobników w oparciu o ich gatunek, poprzez np. wyznaczanie funkcji przystosowania gatunku, czy liczby osobników, które ten gatunek stworzy w kolejnej generacji.
- Dane o wszystkich połączeniach i węzłach, jakie pojawiły się w toku działania algorytmu, znajdują się w obiekcie klasy InnovationTracker. Klasa ta posiada dwa kontenery asocjacyjne, które służą do zapisywania historii połączeń i węzłów:
  - Dla połączeń, kluczem kontenera asocjacyjnego są identyfikatory dwóch węzłów, które to połączenie łączy.
  - Dla węzłów, kluczem kontenera asocjacyjnego jest połączenie, z którego te węzły zostały stworzone za pomocą mutacji (rozdział 3.5). Dane o węzłach zawierają także dane o dwóch połączeniach (struktura NodeGenerationRecord), które powstały podczas mutacji polegającej na dodaniu danego węzła.
- Wszystkie osobniki (obiekty klasy Specimen) przechowywane są kontenerze w obiekcie klasy Population. Klasa ta służy do specjalizacji i reprodukcji osobników w populacji, a także do aktualizowania gatunków.
- Klasa NEAT inicjuje wszystkie elementy implementacji algorytmu i jest odpowiedzialna za uruchomienie całego algorytmu.

Diagram klas i struktur implementacji algorytmu NEAT znajduje się na rysunku 4.6.



Rysunek 4.6. Diagram klas i struktur powiązanych z implementacją algorytmu NEAT.

4.4.3. Architektura programu trenującego

Klasa abstrakcyjna Trainer służy do zdefiniowania sposobu ewaluacji i wyznaczania funkcji przystosowania osobników. Posiada tylko jedną metodę — *evaluate*.

Do trenowania sztucznej inteligencji do gry służy klasa HOMM3BattleTrainer. Obiekt tej klasy ewaluuje osobniki według założeń opisanych w rozdziale 5.5. Ewaluacja ta jest oparta o rywalizację pomiędzy osobnikami w bitwach rozgrywanymi w środowisku gry. Aby przyspieszyć działanie programu, stworzono implementację puli wątków, opierając się na otwartoźródłowej implementacji wzorca(ang. Thread pool)[12]. Pula wątków umożliwia przeprowadzanie wielu pojedynków między osobnikami jednocześnie (listing 2).

Listing 2. Wyznaczanie funkcji przystosowania osobników.

```
1  void HOMM3BattleTrainer::evaluate(){
2
3  vector<BattleScenario> scenarios;
4  // Tworzenie scenariuszy... (opisane w rozdziale 5.4)
5  unordered_map<int, float> points;
6  unordered_map<int, mutex> score_mutexes;
7  // Inicjacja map...
8  vector<int> population_ids = getSpecimensIDs();
9  // Tworzenie wszystkich możliwych kombinacji dwóch osobników z populacji.
10 vector<pair<int, int>> battle_pairs =
11     generateBattlePairs(population_ids);
12
13 for(const auto& [specimen_1_id, specimen_2_id]: battle_pairs){
14     Specimen& specimen_1 = getSpecimenWithID(specimen_1_id);
15     float& model_1_score = points.at(specimen_1_id);
16     mutex& model_1_mutex = score_mutexes.at(specimen_1_id);
17     // Tworzenie elementów dla drugiego osobnika...
18
19     for(BattleScenario& scenario: scenarios){
20         enqueueBattle(specimen_1, specimen_2, scenario,
21         model_1_score, model_2_score,
22         model_1_mutex, model_2_mutex);};}
23
24 pool_.waitForAll();
25 for(const auto& id: population_ids){
26     float wins = points[id];
27     getSpecimenWithID(id).addFitness(wins);}
28
29 }
```

5. Architektura komputerowego gracza

Jednym z celów tej pracy jest stworzenie komputerowego gracza, który jest w stanie zarządzać armią w trakcie fazy bitwy w stworzonej dla tej pracy uproszczonej wersji gry *Heroes of Might and Magic 3*. Głównym zadaniem podczas tworzenia komputerowego agenta jest wytrenowanie za pomocą algorytmu NEAT sieci neuronowej, która byłaby w stanie zarządzać ruchami stosów na polu bitwy.

5.1. Reprezentacja stanu gry

W tej pracy, wzorując się na innej pracy o sztucznej inteligencji w strategicznych grach turowych[13], zrezygnowano z przekazywania informacji dotyczących położenia stosów względem pola bitwy, a zamiast tego zdecydowano się dołączyć do stanu gry informację o relatywnych położeniach stosów względem stosu wykonującego w danym momencie ruch. Takie podejście umożliwia przekazanie sztucznej inteligencji dokładniejszej i bardziej zrozumiałej informacji o relacjach między stosami, niż w przypadku przekazywania informacji o pozycjach stosów względem pola bitwy. Innymi informacjami w wektorze stanów dotyczącymi stosów na polu bitwy są m.in. ich liczba punktów zdrowia, liczba jednostek w stosie, czy średnia liczba zadawanych punktów obrażeń. W wektorze stanu, oprócz informacji o stosach, znajdują się także informacje o bohaterach obydwu armii oraz o posiadanych przez nich zaklęciach.

W celu stworzenia wektora stanu, zaprojektowano zestawy zmiennych, które opisują stan poszczególnych elementów bitwy oraz pola walki. W tabeli 5.1 opisano zmienne dla obiektów będących przyjaznymi stosami, w tabeli 5.2 zmienne dla wrogich stosów, w tabeli 5.3 zmienne dla bohaterów, a w tabeli 5.4 zmienne dla zaklęć. Całkowita reprezentacja stanu, jaka jest przekazywana komputerowemu agentowi, jest przedstawiona w tabeli 5.5. Wszystkie wartości w wektorze stanu gry są zależne od stosu wykonującego w danym momencie ruch. Wszystkie zmienne opisujące stan gry są także liczbami nieujemnymi.

Dane w wektorze stanów są normalizowane — każda z wielkości jest dzielona przez pewną stałą powiązaną z rodzajem informacji przez nią opisywaną. Stałe zostały wyznaczone na dwa sposoby:

- Dla zmiennych, dla których znana jest ich maksymalna możliwa wartość, dzielnik normalizacji jest równy maksymalnej wartości tej zmiennej. Każda z tych zmiennych w wektorze stanu ma wartość z przedziału $[0; 1]$.
- Dla zmiennych, dla których nieznana jest ich maksymalna możliwa wartość, dzielnik normalizacji został wyznaczony na podstawie obserwacji stanów bitew w taki sposób, aby w większości przypadków (ok. 95%), wartości tych zmiennych w wektorze stanu były z przedziału $[0; 1]$. Wartości tych zmiennych mogą być jednak większe.

Reprezentacje stanów stosów są posortowane rosnąco wewnątrz wektora stanu gry na podstawie dystansu, jaki jest między danym stosem a stosem wykonującym w danym

Tabela 5.1. Reprezentacja stanu przyjaznego stosu.

Lp	Opis zmiennej	Znana max. wartość?	Dzielnik normalizacji
1	Dystans (wyrażony jako liczba ruchów stosu potrzebnych do przesunięcia się na dane pole) względem stosu wykonującego teraz ruch	Nie	20
2	Liczba punktów zdrowia jednej jednostki	Tak	200
3	Aktualna liczba jednostek w stosie	Nie	100
4	Zasięg ataku stosu	Tak	12
5	Szybkość stosu	Tak	12
6	Średnia liczba zadawanych przez stos punktów obrażeń (przy założeniu, że cel ataku ma zerową premię do obrony)	Nie	150

Tabela 5.2. Reprezentacja stanu wrogiego stosu.

Lp	Opis zmiennej	Znana max. wartość?	Dzielnik normalizacji
1	Dystans (wyrażony jako liczba ruchów stosu potrzebnych do przesunięcia się na dane pole) względem stosu wykonującego teraz ruch	Nie	20
2	Liczba punktów zdrowia jednej jednostki	Tak	200
3	Aktualna liczba jednostek w stosie	Nie	100
4	Zasięg ataku stosu	Tak	12
5	Szybkość stosu	Tak	12
6	Średnia liczba punktów obrażeń, jaki stos wykonujący teraz ruch może zadać temu stosowi	Nie	200
7	Średnia liczba punktów obrażeń, jaki ten stos może zadać stosowi wykonującemu teraz ruch	Nie	200
8	Flaga, która oznacza, czy stos wykonujący teraz ruch może zaatakować ten stos	Tak	1

Tabela 5.3. Reprezentacja stanu bohatera.

Lp	Opis zmiennej	Znana max. wartość?	Dzielnik normalizacji
1	Suma punktów ataku i punktów morale bohatera	Nie	20
2	Suma punktów obrony i punktów morale bohatera	Nie	20
3	Aktualna liczba punktów many bohatera	Nie	150
4	Szybkość bohatera	Tak	12

momencie ruch. Stosy są także pogrupowane względem tego, czy dany stos jest przyjazny czy wrogi względem stosu wykonującego w danym momencie ruch.

Stan zaklęcia zawiera informacje o wszystkich statystykach, jakie zaklęcie może posia-

Tabela 5.4. Reprezentacja stanu zaklęcia.

Lp	Opis zmiennej	Znana max. wartość?	Dzielnik normalizacji
1	Koszt rzucenia zaklęcia	Tak	32
2	Liczba zadawanych przez zaklęcie punktów obrażeń	Nie	500
3	Promień działania zaklęcia	Tak	8
4	Liczba punktów ataku, które to zaklęcie dodaje do statystyk przyjaznych stosów	Tak	12
5	Liczba punktów obrony, które to zaklęcie dodaje do statystyk przyjaznych stosów	Tak	12

Tabela 5.5. Reprezentacja stanu bitwy przekazywana wejściu sieci neuronowej.

Opis obiektu	Liczba wartości opisujących stan obiektu
Obiekty przyjazne względem stosu wykonującego w danym momencie ruch	
Bohater	4
Zaklęcie bohatera numer 1	5
...	
Zaklęcie bohatera numer 4	5
Stos wykonujący teraz ruch	6
Stos, który jest najbliżej stosu wykonującego w danym momencie ruch	6
...	
Stos, który jest najdalej (jest siódmy w kolejności) względem stosu wykonującego w danym momencie ruch	6
Obiekty wrogie względem stosu wykonującego w danym momencie ruch	
Bohater	4
Zaklęcie bohatera numer 1	5
...	
Zaklęcie bohatera numer 4	5
Stos, który jest najbliżej stosu wykonującego w danym momencie ruch	8
...	
Stos, który jest najdalej (jest ósmy w kolejności) względem stosu wykonującego w danym momencie ruch	8
Łączna liczba zmiennych składająca się na stan gry	**160**

dać — nawet, jeśli nie wykorzystuje ono danej statystyki (wówczas wartość takiej statystyki jest równa 0). Dzięki takiemu podejściu, reprezentacja stanu zaklęcia posiada informacje o typie zaklęcia — przykładowo, jeśli promień działania zaklęcia jest niezerowy, to takie zaklęcie jest zaklęciem działającym obszarowo.

Jeśli liczba obiektów danego typu jest mniejsza od maksymalnej możliwej liczby tych

obiektów, w wektorze stanu, w miejscach, gdzie powinny znajdować się dane o brakujących obiektach, znajduje się liczba 0, dzięki czemu wektor stanu gry ma zawsze taką samą liczbę elementów.

5.2. System sterowania ruchami stosu

Gracz sterujący armią podczas bitwy w grze *HOMM3*, w celu wykonania akcji stosem, musi wybrać pole, na które powinien udać się stos wykonujący teraz ruch, albo wybrać wrogi stos, który ma zostać zaatakowany. Podejście, w którym agent wykorzystujący sztuczną inteligencję wybiera w ramach swojej tury konkretne pole, na które powinien udać się stos, lub wybiera przeciwnika, którego ma zaatakować, może okazać się bardzo trudne do nauczenia. Wynika to z konieczności przyswojenia przez agenta złożonych informacji o polu bitwy i o relacjach między położonymi na nim stosach. Dodatkowo, stan gry dostępny dla sztucznej inteligencji w tej pracy nie zawiera danych o położeniach stosów względem pola bitwy. Wobec tego, zastosowano inny sposób wyznaczania ruchu agenta wykorzystującego sztuczną inteligencję.

Często w przypadku tworzenia sztucznej inteligencji do gier, stosowanym sposobem reprezentacji ruchu komputerowego gracza jest stosowanie heurystyk[14]. Dzięki takiemu podejściu, zadaniem sztucznej inteligencji będzie nauczenie się zastosowywania odpowiedniej heurystyki w oparciu o stan gry, co powinno umożliwić jej odpowiednie zarządzanie stosami bez potrzeby dokładnego wyznaczania celu na daną turę.

W ramach tej pracy, opracowano zestaw heurystyk, za pomocą których komputerowi agenci są w stanie wykonywać swoje ruchy. Zastosowany zestaw heurystyk przedstawia tabela 5.6.

Tabela 5.6. Zestaw heurystyk dostępnych dla sztucznej inteligencji.

Lp	Nazwa heurystyki	Opis heurystyki
1	Idź naprzód	Na początku walki, wszystkie stosy jednostek należące do danej armii znajdują się po jednej stronie pola bitwy. Ta heurystyka sprawia, że stos, który wykonuje teraz ruch, przesunie się w kierunku przeciwnika.
2	Ukryj stos	Idź do takiego pola, dla którego suma ataków wrogich stosów, które byłyby w stanie zaatakować stos wykonujący teraz ruch, jest najmniejsza.
3	Idź w kierunku przyjaznych stosów	Idź do takiego pola, dla którego średnia odległość (w kartezjańskim układzie współrzędnych) od wszystkich przyjaznych stosów jest najmniejsza.

4	Chroń najsilniejszy stos	Idź do pola, które w najlepszy sposób uniemożliwi wrogim stosom atak na przyjazny stos z największym atakiem. W tej heurystyce wybierane jest pole, dla którego suma ataków wrogich stosów, które byłyby w stanie zaatakować najsilniejszy przyjazny stos, przy założeniu, że stos wykonujący teraz ruch udał się na to pole, jest najmniejsza. Jeżeli stos, który wykonuje teraz ruch, jest jednocześnie najsilniejszym stosem, zamiast tej heurystyki wykonywana jest heurystyka "Ukryj stos".
5	Oddal się od przyjaznych stosów	Idź do takiego pola, którego średnia odległość (w kartezjańskim układzie współrzędnych) od przyjaznych stosów jest największa.
6	Zaatakuj wroga, który ma największą liczbę punktów zdrowia	Zaatakuj wrogi stos, który ma największą liczbę punktów zdrowia.
7	Zaatakuj wroga z najmniejszą liczbą punktów ataku	Zaatakuj wrogi stos, który może zadać najmniejszą średnią liczbę punktów obrażeń.
8	Zaatakuj wroga, aby najbardziej zmniejszyć atak przeciwnej armii	Dla każdego wrogiego stosu W, który da się zaatakować w tej turze, oblicz stratę ataku W, gdyby został on zaatakowany przez stos wykonujący w danym momencie ruch. Strata punktów ataku jest obliczana jako iloczyn średniej liczby zadawanych przez jedną jednostkę w W punktów obrażeń i liczby jednostek, które W utraciłby w wyniku ataku. Zaatakuj taki wrogi stos, dla którego strata ataku będzie największa.
9	Zniszcz wrogi stos z największym atakiem	Dla każdego wrogiego stosu W, który można zaatakować w tej turze, sprawdź, czy atak stosem wykonującym teraz ruch zakończyłby się stratą wszystkich jednostek W. Zaatakuj wrogi stos, który spełnia ten warunek i który może zadać największą średnią liczbę punktów obrażeń.
10	Zaatakuj wroga z największym zasięgiem	Zaatakuj wrogi stos, który ma największy zasięg ataku.

11	Zaatakuj wroga, który może zniszczyć moje jednostki	Dla każdego wrogiego stosu W, który da się zaatakować w tej turze, sprawdź, czy istnieje co najmniej jeden przyjazny stos P, który może zostać pozbawiony przez W wszystkich jednostek. Zaatakuj wroga, który spełnia ten warunek i który może zniszczyć przyjazny stos z największą średnią liczbą zadawanych punktów obrażeń.
12	Zaatakuj wroga, który może najbardziej znacząco zmniejszyć atak mojej armii	Dla każdego wrogiego stosu W oblicz, jaką stratę ataku może ponieść przyjazna armia, gdyby W zaatakował teraz zgodnie z heurystyką "Zaatakuj wroga, aby najbardziej zmniejszyć atak wrogiej armii". Zaatakuj wroga, dla którego strata ataku przyjaznej armii byłaby największa.
13	Zaatakuj wroga, który jest następny w kolejce do ataku	Zaatakuj wrogi stos, który jest następny w kolejce do wykonania swojego ruchu.
14	Nie podejmuj żadnej akcji	Nie wykonuj żadnej akcji stosem wykonującym teraz ruch; stos pozostaje na tym samym polu.

Zestaw heurystyk zastosowanych do trenowania sztucznej inteligencji jest podzielony na dwa zestawy:

- Heurystyki, które kończą się atakiem — którymi są heurystyki oznaczone numerami od 6 do 13.
- Heurystyki, które nie kończą się atakiem — pozostałe heurystyki.

W przypadku heurystyk kończących się atakiem, jako cel najpierw wybierane są takie wrogie stosy, które są w zasięgu ataku stosu wykonującego teraz ruch. Jeśli nie ma takiego stosu, stos wykonujący w danym momencie ruch będzie próbował zbliżyć się do takiego wrogiego stosu, dla którego ocena ataku przez heurystykę jest najlepsza. Jeśli stos nie będzie mógł zbliżyć się do wrogów (np. jest całkowicie otoczony przez przyjazne stosy, co uniemożliwia mu jakikolwiek ruch), wówczas, w zależności od heurystyki może albo spróbować wykonać inną heurystykę, albo jego tura zostanie pominięta.

W przypadku heurystyk nie kończących się atakiem, pod uwagę brane są wszystkie pola, na które może udać się w tej turze stos wykonujący teraz ruch. Jeśli żadne pole nie spełnia wymagań stawianych przez heurystykę, stos będzie próbował wykonać inną akcję, albo pominie turę.

Przyjęto, że każde wyjście sieci neuronowej jest powiązane z jedną z wymienionych heurystyk. Zadaniem agenta, jest wybór odpowiedniej heurystyki w zależności od aktualnego stanu gry.

5.3. System rzucania zaklęć

Podczas trenowania sztucznej inteligencji za pomocą algorytmu NEAT, sieć neuronowa uczy się jedynie podejmowania decyzji powiązanych z ruchami stosów. Agent nie uczy się zatem sterowania bohaterem i rzucania zaklęć. Niemniej jednak, sztuczna inteligencja powinna nauczyć się zarządzać stosami w taki sposób, aby móc reagować na zaklęcia rzucane podczas bitwy. Wobec tego, stworzono heurystyczny system rzucania zaklęć. System ten bierze pod uwagę wszystkie dostępne dla bohatera zaklęcia, ocenia wszystkie potencjalne cele tych zaklęć, a następnie wybiera to zaklęcie i cel, które zostały ocenione jako najkorzystniejsze w danej sytuacji. Wzór, na podstawie którego oceniana jest jakość zaklęcia, zależy od jego rodzaju:

- Dla zaklęć wzmacniających przyjazne stosy:

$$ocena(s,p) = \sum_{t \in T_{aff}} a_t \cdot n_s \cdot 0,05, \tag{5}$$

  gdzie s oznacza oceniane zaklęcie, p oznacza pole, które jest celem s, T_{aff} oznacza zbiór wszystkich przyjaznych stosów t, które zostałyby objęte działaniem s rzuconego na p, a_t oznacza średnią liczbę punktów obrażeń zadawanych przez t, a n_s oznacza liczbę punktów statystyk, które są dodawane do statystyk t przez s.

- Dla zaklęć zadających obrażenia wrogim stosom:

$$ocena(s,p) = \sum_{t \in T_{aff}} \left\lfloor \frac{d_s}{h_t} \right\rfloor \cdot a_t, \tag{6}$$

  gdzie s oznacza oceniane zaklęcie, p oznacza pole, które jest celem s, T_{aff} oznacza zbiór wszystkich wrogich stosów t, które zostałyby objęte działaniem s rzuconego na p, d_s oznacza liczbę punktów obrażeń, które zadaje s, h_t oznacza liczbę punktów zdrowia pojedynczej jednostki w t, a a_t oznacza średnią liczbę punktów obrażeń zadawanych przez t.

5.4. Scenariusze bitew

Podczas trenowania sztucznej inteligencji, komputerowi agencji biorą udział w bitwach między sobą. Aby ułatwić inicjację środowiska bitwy, zaimplementowano system tworzenia scenariuszów bitwy. W ramach trenowania sztucznej inteligencji, tworzone są zestawy zasobów, które przypadną każdemu z agentów podczas bitwy. Zestawy zasobów, dla pojedynczej bitwy są takie same dla obydwu jej uczestników.

Przy tworzeniu scenariuszy bitwy używanych podczas trenowania, przyjęto następujące założenia:

- Liczba różnych stosów należących do jednej armii wynosi 8 (jest to maksymalna możliwa liczba stosów w armii).

- Suma wartości SI stosów należących do jednej armii powinna wynosić około 20000.
- Każdy z bohaterów ma do dyspozycji 4 zaklęcia na poziomie 1.
- Każdy z bohaterów otrzymuje 10 dodatkowych punktów statystyk.
- Każdy rodzaj zasobów (bohater, rodzaje jednostek, liczebność stosów, zaklęcia, przydział punktów statystyk), jak i mapa, na której rozgrywana jest bitwa, są tworzone losowo podczas trenowania.



Rysunek 5.1. Przykładowa armia stworzona na potrzeby treningu sztucznej inteligencji.

5.5. Ewaluacja agentów

Do trenowania agentów, zdecydowano się na zastosowanie koewolucji kompetetywnej. W tym rodzaju trenowania sztucznej inteligencji za pomocą algorytmów ewolucyjnych, wartość funkcji przystosowania pojedynczego osobnika jest obliczana na podstawie pewnego rodzaju turnieju przeciwko innym osobnikom w populacji. Takie podejście jest

często wykorzystywane w różnego rodzaju zadaniach z wykorzystaniem algorytmów ewolucyjnych[6][15][16].

Podczas jednej iteracji algorytmu, każdy z osobników (agentów) bierze udział w bitwach przeciwko każdemu z pozostałych osobników wchodzących w skład populacji. Pomiędzy każdą możliwą parą osobników odbywa się 10 walk, opartych na 5 scenariuszach bitew. Każda para osobników bierze udział w dwóch bitwach w ramach jednego scenariusza — tak, aby każdy z graczy mógł wykonać pierwszy ruch podczas bitwy. Za każde zwycięstwo w pojedynczej bitwie, osobnik otrzymuje jeden punkt. Liczba punktów zdobytych przez osobnika podczas wszystkich bitew w ramach jednej generacji stanowi jego wartość funkcji przystosowania.

5.6. Walidacja sztucznej inteligencji

Korzystając wyłącznie z koewolucji kompetetywnej, trudno jest ustalić, czy sztuczna inteligencja rzeczywiście wytworzyła dobrą strategię zarządzania armią. Dlatego zdecydowano się na wprowadzenie innego rodzaju gracza komputerowego, niewykorzystującego wytrenowanych sieci neuronowych, w celu walidacji agentów stworzonych za pomocą algorytmu NEAT.

Implementacja nowego gracza jest oparta o projekt VCMI[8] – otwartoźródłowy projekt stworzony przez graczy *HOMM3*, którego celem jest odtworzenie oryginalnego silnika gry w celu rozszerzenia go o nowe funkcjonalności[17]. Agent wykorzystywany w tej pracy jest uproszczoną wersją sztucznej inteligencji wykorzystywanej w powyższym projekcie[18]. Algorytm ten, dla każdego wroga w zasięgu ataku stosu wykonującego teraz ruch, przeprowadza symulację ataku na niego oraz zadanej liczby kolejnych tur bitwy (liczba tur jest jednocześnie „poziomem trudności” gracza), przy czym dla dalszych tur symulacji, algorytm podejmuje akcje w sposób zachłanny, za każdym razem wybierając ruchy według zasady "Zaatakuj wroga, aby najbardziej zmniejszyć atak przeciwnej armii"(heurystyka nr 8 w tabeli 5.6). Algorytm na końcu wybiera ten atak, dla którego symulacja walki dała najlepszy potencjalny zysk w bitwie. Zysk jest obliczany na podstawie różnicy w sumach ataków stosów pomiędzy armią przyjazną i wrogą. Ocena ataku jest tym lepsza, im różnica między sumą ataków armii po przeprowadzeniu symulacji jest większa (na korzyść przyjaznej armii). Zastosowanie gracza opartego na projekcie VCMI do walidacji komputerowych graczy pozwala nie tylko na obiektywne porównanie stworzonych modeli, ale także umożliwia ocenę skuteczności tworzenia agenta przy użyciu metod uczenia maszynowego w porównaniu z graczem opartym o zestaw reguł opracowanych przez człowieka.

[8] VCMI jest akronimem wyrażenia *Vinyar Callor Meletyo ar Ingoleo*, który oznacza w *języku quenejskim* (sztucznym języku opracowanym przez J.R.R. Tolkiena na potrzeby jego książek) "Nowy *Heroes of Might and Magic*"

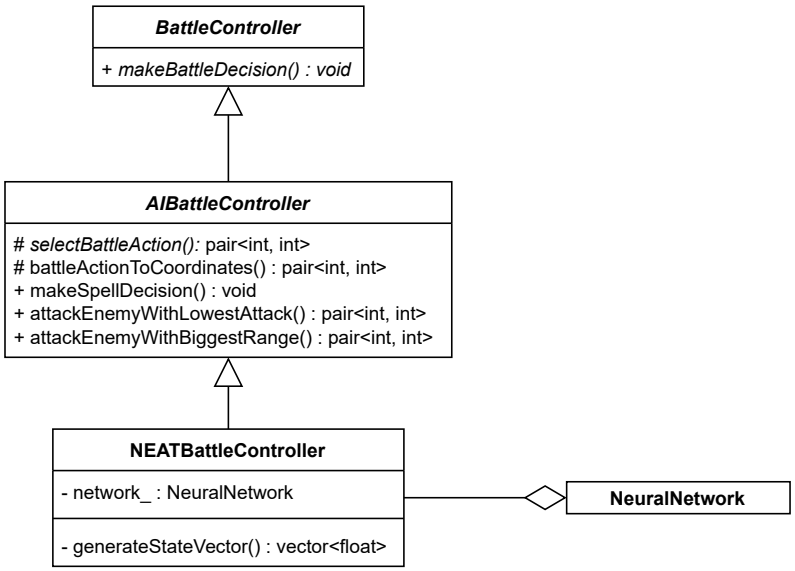
5.7. Implementacja komputerowego gracza

Implementacją komputerowego gracza, wykorzystującego sieć neuronową wytrenowaną za pomocą algorytmu NEAT, jest klasa NEATBattleController (rysunek 5.2). Klasa ta posiada metodę *generateStateVector*, która generuje wektor stanu, a za pomocą obiektu klasy NeuralNetwork, kontroler podejmuje decyzje o akcjach stosu. Klasa NEATBattleController implementuje klasę AIBattleController, która stanowi interfejs między kontrolerami wykorzystującymi heurystyki do podejmowania akcji (tabela 5.6), a klasą BattleController, która jest wykorzystywana w silniku gry do sterowania zasobami podczas bitew. Klasa AIBattleController implementuje wykorzystywane przez sztuczną inteligencję heurystyki, a także implementuje system rzucania zaklęć opisany w rozdziale 5.3. AIBattleController udostępnia klasom dziedziczącym po niej wirtualną metodę *selectBattleAction*, która wybiera akcję w danej turze bitwy i której implementacja w klasie NEATBattleController jest przedstawiona w listingu 3.

Instancja klasy NEATBattleController jest wykorzystywana w środowisku gry jako komputerowy agent sterujący neutralnymi armiami znajdującymi się na mapie świata.

Listing 3. Implementacja wyboru akcji w klasie NEATBattleController.

```
1
2  pair<int, int> NEATBattleController::selectBattleAction(){
3      vector<float> state = generateStateVector();
4      vector<float> output = network_.propagate(state);
5
6      auto max_value = max_element(output.begin(), output.end());
7      // AI_BATTLE_ACTIONS jest zbiorem wszystkich heurystyk wykorzystywanych
           przez sztuczną inteligencję do podejmowania akcji.
8      BattleAction decision = AI_BATTLE_ACTIONS.at(
9          distance(output.begin(), max_value));
10
11     //...
12
13     // Metoda battleActionToCoordinates(), zaimplementowana w klasie
           AIBattleController, uruchamia metodę powiązaną z wybraną heurystyką i na
           jej podstawie wyznacza koordynaty, które oznaczają cel akcji.
14     pair<int, int> coordinates =
15         battleActionToCoordinates(decision);
16
17     return coordinates;
18 }
```



Rysunek 5.2. Diagram UML klas NEATBattleController, AIBattleController i klas z nimi powiązanych

.